



KAI

For RAD Studio | Delphi | C++Builder



2026

A P O S T I L A

DESENVOLVIMENTO RÁPIDO. RESULTADOS INTELIGENTES.

A inteligência que transforma
código em produtividade.



IA PARA DESENVOLVEDORES

Gere código, refatore,
explique e muito mais.



INTEGRAÇÃO NATIVA

Conectado ao seu
workflow em RAD,
Delphi e C++Builder.



PRODUTIVIDADE SEM LIMITES

Menos tempo
codando. Mais tempo
criando soluções.

Inteligência Artificial para RAD Studio, Delphi e C++Builder

Do código à automação inteligente.

Aprenda a utilizar o KAI para:

- Gerar código automaticamente
- Explicar projetos complexos
- Refatorar aplicações legadas
- Criar testes unitários
- Integrar IA Local com Ollama
- Utilizar MCP (Model Context Protocol)
- Modernizar aplicações Delphi e C++Builder
- Aumentar a produtividade no desenvolvimento

Sumário

Inteligência Artificial para RAD Studio, Delphi e C++Builder.....	2
Do código à automação inteligente.....	2
MÓDULO 1 - O QUE É O KAI?	8
Objetivos:	8
O que significa KAI?	8
Recursos Principais do KAI	10
Chat Inteligente	10
Ghost Text.....	10
Completions	11
Refatoração Assistida	11
Testes Unitários	11
Benefícios para Delphi	11
Benefícios para C++Builder	12
Modelos de IA Compatíveis	13
IA Local com Ollama	13
Exercícios:	14
Exercício Prático 1	14
Exercício Prático 2.....	14
Exercício Prático 3.....	14
Resumo do Módulo	15
MÓDULO 2 - INSTALAÇÃO, LICENCIAMENTO E CONFIGURAÇÃO DO KAI	16
Introdução	16
Pré-Requisitos	17
IDE Compatível	17
Edição da IDE.....	17
Como Obter o KAI	17
Método 2 – Customer Portal	18
Processo de Instalação.....	18
Informações da Instalação	18
Diretório de Instalação.....	18
Tipos de Instalação	19
Conclusão da Instalação	19
Licenciamento do KAI.....	20
Trial do KAI.....	20
Licença Comercial	20
Ativando a Licença	20

Como Verificar se Está Ativo	20
Configurando o KAI	21
Seções de Configuração.....	21
Chat.....	21
Completions	21
MCP Servers	21
Modelos de IA.....	22
Configurando Ollama	22
Ambiente Corporativo.....	22
Exercício Prático 1	23
Exercício Prático 2.....	23
Exercício Prático 3.....	23
Exercício Prático 4.....	23
Problemas Comuns	23
Licença não encontrada	24
Chat não responde.....	24
Resumo do Módulo	24
MÓDULO 3 - ENGENHARIA DE PROMPT PARA C++BUILDER 13.1 FLORENCE	25
Introdução	25
O Que é um Prompt?	25
Estrutura de um Prompt Profissional.....	26
Exercício 1	26
Exercício 2	27
Como Pedir Explicações?	28
Exercício 3	28
Utilizando Contexto do Editor.....	29
Exercício 4	29
Exercício 5	30
Criando Componentes VCL.....	30
Gerando Código FireDAC	31
Gerando Código REST.....	31
Criando Prompts para Migração	32
Prompts que Funcionam Muito Bem.....	32
Prompts que Devem Ser Evitados.....	33
Exercício Final do Módulo.....	33
Resumo do Módulo.....	34
MÓDULO 4 - GERAÇÃO DE CÓDIGO VCL COM KAI NO C++BUILDER 13.1 FLORENCE. 35	35
Introdução	35
RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0	

Preparação do Ambiente	35
Exercício Inicial.....	35
Primeiro Exemplo.....	36
Tela de Login	36
Melhorando o Resultado	37
Exercício 2	37
Cadastro de Clientes	37
Exercício 3	38
Dashboard Simples	38
Aprendendo a Corrigir o KAI	39
Exercício 4	39
Tela de Configuração	39
Exercício 5	40
Consumo REST.....	40
Exercício 6	40
Consulta JSON	40
Exercício 7	41
CRUD Simulado.....	41
Exercício 8	42
Dashboard Industrial	42
Solicitando Melhorias	42
Solicitando Refatoração	43
Solicitando Documentação	43
Exercício Final.....	44
Criar um Mini ERP.	44
Checklist de Aprovação	44
Problemas Comuns	45
Resumo do Módulo.....	45
MÓDULO 5 - ANALISANDO, ENTENDENDO E MODERNIZANDO PROJETOS REAIS COM KAI	46
Introdução	46
Cenário Real.....	46
O Problema da Manutenção	47
Exercício 1	47
Entendendo uma Classe	47
Exercício 2	48
Explicação Método por Método	48
Exercício 3	48

Descobrimos Regras de Negócio	48
Exemplo Real	49
Exercício 4	49
Mapeamento de Dependências.....	49
Exercício 5	50
Encontrando Problemas	50
Analisando Projetos Grandes	50
Exercício 6	51
Gerando Documentação Técnica	51
Exercício 7	51
Criando Diagramas Textuais	51
Modernização de Código	52
Exercício 8	52
Exercício 9	53
Refatoração Controlada.....	53
Exercício 10	53
Identificando Código Duplicado	53
Analisando Performance.....	53
Segurança	54
Exercício Final.....	54
Cenário Profissional	55
Resumo do Módulo.....	56
MÓDULO 6 - MIGRAÇÃO ASSISTIDA DE PROJETOS C++BUILDER 6 PARA C++BUILDER	
13.1 FLORENCE UTILIZANDO KAI	57
Processo Utilizando KAI.....	58
Fase 1 - Inventário do Projeto.....	59
Exercício 1	59
Selecione uma Unit.....	59
Exercício 2	60
Selecione o arquivo principal.	60
Fase 2 - Compatibilidade com Florence	60
Problema 1 - Unicode.....	61
Problema 2 - APIs ANSI.....	61
Problema 3 - Conversões de Texto	62
Fase 3 - Compatibilidade com Clang.....	62
Exercício 3	62
Selecione uma classe.	62
Fase 4 - Componentes de Terceiros	63

Exercício 5	63
Identifique componentes de terceiros utilizados nesta Unit.	63
Fase 5 - Planejamento da Migração.....	63
Fase 6 - Geração de Relatório	64
Exercício Final - Utilize um projeto real.....	65
Estudo de Caso - Migração Assistida	66
Boas Práticas –	66
Encerramento do Curso	67

MÓDULO 1 - O QUE É O KAI?

Objetivos:

Ao final deste módulo o aluno será capaz de:

- Entender o conceito do KAI
- Compreender o papel da IA dentro do RAD Studio
- Identificar os principais benefícios para Delphi e C++Builder
- Entender a arquitetura básica do KAI
- Conhecer os recursos disponíveis na versão Florence

Introdução

Durante muitos anos os desenvolvedores utilizaram ferramentas de produtividade tradicionais:

- Code Insight
- Auto Complete
- Templates
- Refactoring
- Wizards

Essas ferramentas aceleravam o desenvolvimento, mas dependiam exclusivamente do conhecimento do programador.

Com a popularização dos Large Language Models (LLMs), surgiram novas possibilidades para auxiliar o desenvolvimento de software.

O KAI representa a entrada da Embarcadero na era da Inteligência Artificial aplicada ao desenvolvimento Delphi e C++Builder.

O que significa KAI?

KAI é a plataforma de Inteligência Artificial integrada ao RAD Studio.

Seu objetivo é transformar a IDE em um ambiente de desenvolvimento assistido por IA permitindo que o desenvolvedor:

- Faça perguntas diretamente na IDE

- Gere código automaticamente
- Explique trechos complexos
- Corrija erros
- Refatore projetos
- Crie testes
- Interaja com ferramentas externas através de MCP

O KAI não substitui o desenvolvedor.

Ele atua como um assistente especializado trabalhando em conjunto com o programador.

O Problema que o KAI Resolve

Antes do KAI era comum o desenvolvedor:

1. Encontrar um problema
2. Abrir o navegador
3. Acessar uma IA externa
4. Copiar código
5. Colar na IDE
6. Ajustar manualmente

Esse processo gera:

- Perda de foco
- Troca constante de contexto
- Menor produtividade
- Maior risco de erros

O KAI elimina grande parte dessas interrupções ao trazer a IA para dentro do ambiente de desenvolvimento.

KAI não é apenas um Chat

Um erro comum é imaginar que o KAI seja apenas uma janela semelhante ao ChatGPT.

Na prática ele é muito mais que isso.

O KAI foi projetado para compreender:

- Arquivos do projeto
- Estrutura da aplicação
- Compilação
- Erros do compilador
- Contexto do código selecionado
- Ferramentas MCP

Isso permite interações muito mais inteligentes do que uma simples conversa.

Recursos Principais do KAI

Chat Inteligente

- Permite conversar com a IA diretamente dentro da IDE.

Exemplos:

"Explique este método"

"Crie uma classe Produto"

"Como funciona FireDAC?"

Ghost Text

- Sugestões *inline* - diretamente no editor.

Enquanto você digita, o KAI pode sugerir:

- Métodos
- Classes
- Propriedades
- Estruturas completas

Completions

- Sugestões avançadas baseadas no contexto do código.

O objetivo é acelerar o desenvolvimento sem sair da IDE.

Refatoração Assistida

O KAI pode auxiliar na modernização de código.

Exemplos:

- Melhorar nomenclaturas
 - Remover duplicação
 - Simplificar algoritmos
 - Atualizar código legado
-

Testes Unitários

Criação automática de:

- Cenários de teste
 - Casos de sucesso
 - Casos de falha
-

Documentação

Geração automática de:

- Comentários
 - Explicações
 - Documentação técnica
-

Benefícios para Delphi

O KAI pode auxiliar em:

- VCL
- FMX

- FireDAC
- REST Client
- JSON
- DataSnap
- RAD Server

Além disso pode explicar códigos Delphi antigos e sugerir modernizações.

Benefícios para C++Builder

Para desenvolvedores C++, o KAI oferece:

- Explicação de ponteiros
- Modernização para Clang
- Conversão de código legado
- Criação de classes
- Integração REST
- FireDAC
- JSON
- IoT

Especialmente útil em processos de migração de versões antigas para Florence.

Arquitetura Simplificada

Fluxo básico:

Desenvolvedor
↓
KAI
↓
Modelo de IA
↓
Resposta
↓
RAD Studio

Em cenários mais avançados:

Desenvolvedor

↓

KAI

↓

MCP

↓

Sistema Externo

↓

Banco de Dados

↓

Dispositivos

↓

Resposta

Modelos de IA Compatíveis

O KAI foi projetado para trabalhar com múltiplos provedores e modelos de IA. A seleção do modelo pode ser configurada dentro da IDE.

Isso oferece flexibilidade para escolher:

- Velocidade
- Custo
- Qualidade da resposta

IA Local com Ollama

Um dos diferenciais mais interessantes é a possibilidade de utilizar modelos locais através do Ollama.

Benefícios:

- Maior privacidade
- Sem envio de código para serviços externos
- Controle total da infraestrutura
- Possibilidade de criar soluções corporativas

Vamos colocar em prática o que aprendemos com o KAI?

Exercícios:

Exercício Prático 1

Abra o Chat do KAI.

Digite:

Explique o que é um ponteiro em C++.

Analise a resposta.

Compare com a documentação oficial.

Exercício Prático 2

Digite:

Crie uma classe Produto em C++Builder.

Observe:

- Estrutura criada
 - Nomenclatura utilizada
 - Organização do código
-

Exercício Prático 3

Selecione um método do seu projeto.

Pergunte:

Explique este código passo a passo.

Analise como o KAI utiliza o contexto selecionado.

Resumo do Módulo

Neste módulo aprendemos:

- O que é o KAI
- Como ele funciona
- Principais recursos
- Benefícios para Delphi
- Benefícios para C++Builder
- Uso de IA Local
- Primeiros exercícios

No próximo módulo veremos a instalação, ativação e configuração completa do KAI dentro do RAD Studio 13.1 Florence.

MÓDULO 2 - INSTALAÇÃO, LICENCIAMENTO E CONFIGURAÇÃO DO KAI

Objetivos

Ao final deste módulo o aluno será capaz de:

- Instalar o KAI no RAD Studio
- Entender o modelo de licenciamento
- Ativar uma licença Trial ou Comercial
- Configurar provedores de IA
- Configurar Chat e Completions
- Verificar se o ambiente está pronto para uso

Introdução

Agora que entendemos o que é o KAI, chegou o momento de colocá-lo para funcionar.

Diferentemente de um simples plugin de IA, o KAI é instalado como um complemento oficial do RAD Studio e passa a fazer parte da experiência da IDE.

Após a instalação, novas funcionalidades passam a estar disponíveis:

- Chat Inteligente
 - Ghost Text
 - Completions
 - Agentes de IA
 - Integração MCP
 - Configurações avançadas de modelos
-

Pré-Requisitos

Antes de instalar o KAI, verifique:

- **Sistema Operacional**

Windows 10 ou superior.

IDE Compatível

- RAD Studio 13 Florence
- Delphi 13 Florence
- C++Builder 13 Florence
-

Também existe suporte para versões 12.x, porém este treinamento utiliza Florence 13.1.

Edição da IDE

O KAI está disponível para:

- Professional
- Enterprise
- Architect

“Qualquer uma destas edições precisa ter MANUTENÇÃO ATIVA (SUBSCRIPTION)”.

A Community Edition não possui suporte ao KAI.

Como Obter o KAI

Existem duas formas principais.

Método 1 – GetIt Package Manager

Na IDE:

Tools

↓

GetIt Package Manager

Pesquise:

"KAI"

Clique em:

Install

Este é o método recomendado pela Embarcadero.

Método 2 – Customer Portal

Outra opção é baixar diretamente através do portal de clientes da Embarcadero utilizando uma conta EDN ativa.

Processo de Instalação

Ao iniciar a instalação serão exibidas algumas telas.

Contrato de Licença

Leia os termos.

Marque:

"I Accept the Agreement"

Clique:

Next

Informações da Instalação

Tela informativa.

Clique:

Next

Diretório de Instalação

Defina o local de instalação.

Normalmente o padrão é suficiente.

Clique:

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

Next

Tipos de Instalação

- Uma das telas mais importantes da instalação.

O KAI permite diferentes perfis.

Full Installation

Instala todos os recursos.

Recomendado para treinamento.

Compact Installation

Instala apenas os recursos principais.

Custom Installation

Permite escolher manualmente.

Local Models Only

Focado em modelos locais.

Ideal para ambientes corporativos.

Chat Only

Instala somente recursos de chat.

Completions Only

Instala apenas sugestões inteligentes.

Conclusão da Instalação

Ao concluir:

Install

↓

Finish

A IDE passará a exibir informações do KAI durante a inicialização.

Licenciamento do KAI

Importante:

O KAI possui licenciamento próprio.

Ele não utiliza a mesma licença do RAD Studio.

São produtos distintos onde a manutenção (subscription) se faz necessária.

Trial do KAI

É possível solicitar uma licença de avaliação.

O período de teste permite explorar os recursos antes da aquisição definitiva.

Licença Comercial

Após a compra você receberá:

- Chave de licença
 - Informações de ativação
-

Ativando a Licença

Abra:

C:\Program Files (x86)\Embarcadero\Studio\37.0\bin

Execute:

LicenseManager.exe

Informe a chave recebida.

Conclua o registro.

Como Verificar se Está Ativo

Abra o RAD Studio.

Observe a Splash Screen.

O KAI exibirá seu estado:

- Active

- Expired
 - Not Found
-

Configurando o KAI

Após a instalação:

Tools



Options



Kai

Seções de Configuração

A área de configuração é dividida em grupos.

Chat

Controla:

- Modelo usado pelo Chat
 - Comportamento do agente
 - Preferências de interação
-

Completions

Controla:

- Ghost Text
 - Sugestões inline
 - Auto completions
-

MCP Servers

Permite registrar:

- Servidores MCP
- Ferramentas corporativas
- Sistemas externos

Modelos de IA

O KAI suporta diferentes provedores.

Exemplos:

- OpenAI
- Gemini
- Claude
- Ollama

A escolha depende da estratégia da empresa.

Configurando Ollama

Uma das configurações mais interessantes.

Vantagens:

- IA local
 - Privacidade
 - Sem envio de código para terceiros
 - Controle corporativo
-

Ambiente Corporativo

Uma arquitetura comum:



Essa arquitetura será explorada nos módulos avançados.

Exercício Prático 1

Instale o KAI utilizando o GetIt.

Checklist:

- Download realizado
 - Instalação concluída
 - IDE reiniciada
-

Exercício Prático 2

Abra:

Tools → Options → Kai

Localize:

- Chat
- Completions
- MCP Servers

Tire screenshots para documentação.

Exercício Prático 3

Verifique a licença.

Confirme se o status aparece como:

Active

Exercício Prático 4

Abra o Chat do KAI.

Pergunte:

"Olá KAI, você está funcionando?"

Valide a resposta.

Problemas Comuns

KAI não aparece

Verifique:

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

- Instalação concluída
 - Reinício da IDE
-

Licença não encontrada

Verifique:

- Registro no License Manager
 - Validade da chave
-

Chat não responde

Verifique:

- Configuração do modelo
 - Conectividade
 - Chave do provedor
-

Resumo do Módulo

Neste módulo aprendemos:

- Como instalar o KAI
- Como ativar a licença
- Como configurar a IDE
- Como selecionar modelos
- Como validar o funcionamento

No próximo módulo aprenderemos a utilizar o Chat do KAI e escrever prompts profissionais para Delphi e C++Builder.

MÓDULO 3 - ENGENHARIA DE PROMPT PARA C++**BUILDER 13.1 FLORENCE**

Objetivos

Ao final deste módulo o aluno será capaz de:

- Criar prompts eficientes para o KAI
- Obter respostas mais precisas
- Gerar código VCL utilizando IA
- Solicitar explicações técnicas detalhadas
- Utilizar contexto do projeto
- Refatorar código através de prompts
- Criar um checklist de validação das respostas do KAI

Introdução

Um dos maiores erros ao utilizar IA é acreditar que basta fazer uma pergunta simples.

Na prática, a qualidade da resposta está diretamente relacionada à qualidade do prompt.

Muitas vezes o desenvolvedor conclui que a IA não funciona quando, na verdade, o problema está na forma como a pergunta foi feita.

O Que é um Prompt?

Prompt é a instrução enviada ao KAI.

Exemplo simples:

"Crie um cadastro de clientes."

O KAI consegue responder.

Mas a resposta pode ser genérica.

Agora veja:

"Crie uma tela VCL para cadastro de clientes utilizando TEdit, TLabel, TButton e FireDAC para SQLite."

A resposta tende a ser muito mais próxima do esperado.

Regra de Ouro

Quanto mais contexto você fornecer, melhor será a resposta.

Sempre informe:

- Linguagem
 - Framework
 - Objetivo
 - Banco de dados
 - Componentes desejados
 - Resultado esperado
-

Estrutura de um Prompt Profissional

Utilize o seguinte modelo:

Linguagem:

C++Builder 13.1 Florence

Tecnologia:

VCL

Objetivo:

Cadastrar clientes

Banco:

SQLite

Componentes:

TEdit

TButton

TLabel

Resultado: Classe completa compilável

Exercício 1

Prompt Simples

Digite no Chat do KAI:

Crie uma tela de cadastro de clientes.

Checklist

- O KAI criou uma classe?
 - O código compila?
 - Foram utilizados componentes VCL?
 - Existe tratamento de erros?
 - Há comentários explicativos?
-

Exercício 2

Prompt Profissional

Digite:

Você é um especialista em C++Builder 13.1 Florence.

Crie uma tela VCL para cadastro de clientes.

Utilize:

- TLabel
- TEdit
- TButton

Crie os métodos:

Salvar

Excluir

Pesquisar

O código deve ser compatível com Clang.

Inclua comentários.

Checklist

- Classe criada
 - Métodos criados
 - Comentários presentes
 - Compilação sem erros
 - Organização adequada
-

Comparação dos Resultados

Prompt Simples:

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

Pouco contexto



Resposta genérica

Prompt Profissional:

Muito contexto



Resposta especializada

Como Pedir Explicações?

O KAI pode atuar como professor.

Exemplo:

Explique ponteiros em C++Builder para um iniciante.

Checklist

- Explicação clara
- Exemplos práticos
- Uso de objetos VCL
- Código demonstrativo

Exercício 3

Solicite:

Explique a diferença entre:

*TButton *Botao*

e

TButton Botao

Validação

Verifique se o KAI explicou:

- Ponteiro
- Instância

- Memória
 - Objetos VCL
-

Utilizando Contexto do Editor

Uma das maiores vantagens do KAI.

Selecione um método existente.

Clique:

Ask Kai

Digite:

Explique este método linha por linha.

Checklist

- O método selecionado foi identificado
 - O contexto foi utilizado
 - O KAI não respondeu genericamente
 - A explicação está correta
-

Exercício 4

Crie uma Unit simples.

Exemplo:

```
class TCalculadora
{
public:
int Somar(int A, int B);
};
```

Selecione a classe.

Pergunte:

Explique esta classe para um desenvolvedor iniciante.

Geração de Métodos

O KAI pode criar implementações automaticamente.

Exemplo:

Crie a implementação da classe abaixo.

[cole a classe]

Checklist

- Métodos implementados
 - Código organizado
 - Compatível com Clang
 - Sem dependências inexistentes
-

Como Pedir Refatoração

Exemplo:

Refatore este código para padrões modernos de C++.

Exercício 5

Cole:

```
char Nome[50];
```

Pergunte:

Converta para uma abordagem moderna utilizando String.

Checklist

- Uso de String
 - Código mais seguro
 - Explicação da mudança
-

Criando Componentes VCL

Prompt:

Crie um exemplo utilizando:

TButton

TEdit

TMemo

Ao clicar no botão, o conteúdo do TEdit deve ser enviado para o TMemo.

Checklist

- Evento OnClick criado
 - Compila
 - Componentes corretos
 - Código comentado
-

Gerando Código FireDAC

Prompt:

Crie uma conexão FireDAC para SQLite em C++Builder 13.1 Florence.

Checklist

- TFDConnection
 - Driver SQLite
 - Tratamento de erros
 - Código comentado
-

Gerando Código REST

Prompt:

Crie um exemplo utilizando TNetHTTPClient para consumir uma API REST.

Checklist

- Headers
 - GET
 - Tratamento de exceções
 - Resposta armazenada
-

Criando Prompts para Migração

Este será um dos recursos mais utilizados.

Prompt:

Analise este código legado do C++Builder 6.

Liste:

- incompatibilidades
 - riscos
 - melhorias
 - compatibilidade com Florence
-

Checklist

- Unicode
 - Clang
 - Componentes antigos
 - APIs depreciadas
 - Sugestões de melhoria
-

Prompts que Funcionam Muito Bem

Explicação

Explique este código linha por linha.

Refatoração

Refatore utilizando boas práticas modernas.

Performance

Otimize este método.

Segurança

Analise possíveis vulnerabilidades.

Migração

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

Analise compatibilidade com Florence.

Prompts que Devem Ser Evitados

Ruim:

Corrija meu sistema.

Ruim:

Crie um ERP.

Ruim:

Melhore isso.

Sempre forneça contexto.

Exercício Final do Módulo

Crie um projeto VCL novo.

Solicite ao KAI:

Crie uma calculadora simples utilizando:

- *TEdit*
- *TButton*
- *TLabel*

Implemente:

Somar

Subtrair

Multiplicar

Dividir

Com tratamento para divisão por zero.

Checklist de Aprovação

- Projeto criado
- Código gerado pelo KAI

- Compilação realizada
 - Interface funcional
 - Operações funcionando
 - Tratamento de erro implementado
 - Código entendido pelo aluno
-

Resumo do Módulo

Neste módulo aprendemos:

- O que é um prompt
- Como criar prompts profissionais
- Como utilizar contexto do editor
- Como solicitar explicações
- Como gerar código
- Como refatorar projetos
- Como validar respostas do KAI

No próximo módulo aprenderemos a utilizar o KAI para geração de código VCL profissional dentro do C++Builder 13.1 Florence.

MÓDULO 4 - GERAÇÃO DE CÓDIGO VCL COM KAI NO C++BUILDER 13.1 FLORENCE

Objetivos

Ao final deste módulo o aluno será capaz de:

- Gerar aplicações VCL utilizando o KAI
- Criar formulários completos
- Gerar eventos automaticamente
- Solicitar melhorias de layout
- Gerar CRUDs básicos
- Criar dashboards simples
- Validar a qualidade do código gerado

Introdução

Uma das funcionalidades mais impressionantes do KAI é a geração de código.

Mas é importante entender:

O KAI não é um gerador mágico de sistemas.

Ele é um acelerador de desenvolvimento.

Quanto melhor for o prompt, melhor será o resultado.

Neste módulo vamos construir aplicações VCL reais.

Preparação do Ambiente

Exercício Inicial

Criar um **novo projeto**:

File

↓

New

↓

VCL Forms Application - C++

Salvar como:

KaiVCLTraining

Checklist

- Projeto criado
 - Form principal aberto
 - Compilação realizada
 - Projeto executado
-

Primeiro Exemplo

Tela de Login

Digite no Chat do KAI:

Crie uma tela VCL de login utilizando:

- *TLabel*
- *TEdit*
- *TButton*

Campos:

Usuário

Senha

Botão:

Entrar

Ao clicar em Entrar exibir ShowMessage informando sucesso.

Utilize C++Builder 13.1 Florence.

O que Avaliar

Interface

- Labels criadas
 - Campos criados
 - Botão criado
-

Código

- Evento OnClick criado
 - Código compilável
 - Comentários presentes
-

Melhorando o Resultado

Agora envie:

Melhore o layout da tela utilizando alinhamento profissional.

Centralize os componentes.

Utilize boas práticas de UX.

Checklist

- Layout mais organizado
 - Espaçamento correto
 - Interface profissional
-

Exercício 2

Cadastro de Clientes

Prompt:

Crie uma tela VCL para cadastro de clientes.

Campos:

Nome

Telefone

Email

Botões:

Novo

Salvar

Excluir

Pesquisar

Utilize componentes VCL nativos.

Crie os eventos dos botões.

Checklist

Campos

- Nome
- Telefone
- Email

Botões

- Novo
- Salvar
- Excluir
- Pesquisar

Eventos

- Todos implementados
-

Exercício 3

Dashboard Simples

Prompt:

Crie uma tela VCL simulando um dashboard.

Componentes:

- *TPanel*
- *TLabel*
- *TProgressBar*

Informações:

CPU

Memória

Disco

Atualização via TTimer.

Checklist

- Dashboard criado
 - TTimer criado
 - Atualização automática
 - Interface organizada
-

Aprendendo a Corrigir o KAI

Nem sempre o código será perfeito.

Exemplo:

O KAI criou um componente inexistente.

Solicite:

Substitua componentes incompatíveis por componentes VCL nativos do C++Builder 13.1 Florence.

Checklist

- Código corrigido
 - Componentes válidos
 - Projeto compila
-

Exercício 4

Tela de Configuração

Prompt:

Crie uma tela de configuração.

Utilize:

- *TCheckBox*
- *TComboBox*

- *TEdit*

Crie um botão Salvar.

Ao clicar, exiba os valores configurados.

Checklist

- CheckBox funcionando
 - ComboBox funcionando
 - Edit funcionando
 - Evento criado
-

Exercício 5

Consumo REST

Prompt:

Crie uma tela VCL utilizando:

- *TEdit*
- *TButton*
- *TMemo*
- *TNetHTTPClient*

Ao clicar no botão:

Consumir uma API REST.

Exibir retorno no TMemo.

Checklist

- TNetHTTPClient criado
 - Requisição GET
 - Tratamento de exceções
 - Resultado exibido
-

Exercício 6

Consulta JSON

Prompt:

Crie um exemplo em C++Builder.

Receba um JSON.

Leia:

Nome

Cidade

Telefone

Exiba em um TMemo.

Utilize System.JSON.

Checklist

- Parse do JSON
 - Leitura dos campos
 - Exibição correta
-

Exercício 7

CRUD Simulado

Prompt:

Crie um CRUD em memória utilizando:

std::vector

Classe:

TCliente

Operações:

Inserir

Alterar

Excluir

Pesquisar

Sem banco de dados.

Checklist

- Classe criada
- Vector criado
- Operações funcionando

Exercício 8

Dashboard Industrial

Este exercício servirá de base para os próximos módulos.

Prompt:

Crie uma tela VCL simulando uma esteira industrial.

Utilize:

- *TShape*
- *TPanel*
- *TLabel*
- *Timer*

Estados:

Parada

Normal

Alta Produção

Cores:

Vermelho

Amarelo

Verde

Checklist

- Estados criados
- Alteração de cores
- Interface funcional

Solicitando Melhorias

Após gerar qualquer tela:

Pergunte:

Como melhorar este layout utilizando conceitos modernos de UX?

Checklist

- Sugestões coerentes
 - Melhorias aplicáveis
 - Componentes existentes
-

Solicitando Refatoração

Prompt:

Refatore este formulário utilizando boas práticas de C++Builder.

Melhore:

- *Legibilidade*
 - *Organização*
 - *Nomes de métodos*
 - *Tratamento de erros*
-

Checklist

- Código mais limpo
 - Métodos organizados
 - Comentários úteis
-

Solicitando Documentação

Prompt:

Documente toda esta Unit.

Inclua:

- *Objetivo da classe*
 - *Métodos*
 - *Eventos*
 - *Fluxo de execução*
-

Checklist

- Comentários gerados
- Métodos documentados
- Fácil entendimento

Exercício Final

Criar um Mini ERP.

Solicite:

Crie uma aplicação VCL composta por:

Tela Principal

Menu:

Clientes

Produtos

Pedidos

Cada opção deve abrir um formulário específico.

Utilize apenas componentes VCL.

Checklist de Aprovação

Estrutura

- Form Principal
 - Form Clientes
 - Form Produtos
 - Form Pedidos
-

Código

- Compila
 - Navegação funcionando
 - Eventos funcionando
-

Qualidade

- Código organizado
 - Comentado
 - Fácil manutenção
-

Problemas Comuns

O KAI gera Delphi

Prompt corretivo:

Responda apenas utilizando C++Builder.

Não utilize Delphi.

O KAI utiliza componentes inexistentes

Prompt corretivo:

Utilize apenas componentes disponíveis na VCL do C++Builder 13.1 Florence.

Código incompleto

Prompt corretivo:

Continue a implementação a partir do ponto interrompido.

Resumo do Módulo

Neste módulo aprendemos:

- Como gerar telas VCL
- Como criar dashboards
- Como gerar CRUDs
- Como consumir APIs
- Como utilizar JSON
- Como corrigir respostas do KAI
- Como solicitar melhorias e refatorações

No próximo módulo iremos conectar o KAI ao código existente do projeto para realizar análise, explicação e modernização de aplicações reais em C++Builder.

MÓDULO 5 - ANALISANDO, ENTENDENDO E MODERNIZANDO PROJETOS REAIS COM KAI

Objetivos

Ao final deste módulo o aluno será capaz de:

- Utilizar o KAI para compreender projetos existentes
 - Analisar Units complexas
 - Entender Classes desconhecidas
 - Identificar dependências
 - Encontrar problemas de arquitetura
 - Modernizar código legado
 - Preparar aplicações para migração
-

Introdução

Na prática, a maior parte do trabalho de um desenvolvedor não é criar sistemas novos.

O mais comum é:

- Corrigir bugs
- Evoluir sistemas existentes
- Dar manutenção
- Migrar aplicações antigas
- Entender código criado por outros desenvolvedores

É justamente neste cenário que o KAI se torna extremamente valioso.

Cenário Real

Imagine que você acabou de assumir um projeto.

O sistema possui:

- 500 Units
- 300 Forms

- 20 anos de desenvolvimento
- Diversos programadores envolvidos

A primeira dificuldade normalmente não é programar.

A primeira dificuldade é entender.

O Problema da Manutenção

Quando recebemos um projeto antigo normalmente encontramos:

- Métodos enormes
- Variáveis sem padrão
- Poucos comentários
- Dependências ocultas
- Regras de negócio espalhadas

Antes do KAI o desenvolvedor precisava gastar horas lendo código.

Hoje podemos acelerar esse processo.

Exercício 1

Entendendo uma Classe

Abra uma Unit qualquer.

Selecione a classe inteira.

Pergunte ao KAI:

Explique esta classe como se eu fosse um novo desenvolvedor do projeto.

Liste:

- Responsabilidade
- Métodos principais
- Dependências
- Possíveis riscos

Checklist

- Identificou corretamente a responsabilidade
 - Explicou os métodos
 - Encontrou dependências
 - Apontou riscos
-

Exercício 2

Explicação Método por Método

Selecione um método grande.

Pergunte:

Explique este método linha por linha.

Mostre:

- *Entrada*
 - *Processamento*
 - *Saída*
-

Checklist

- Fluxo explicado
 - Variáveis identificadas
 - Resultado compreendido
-

Exercício 3

Descobrimo Regras de Negócio

Pergunte:

Quais regras de negócio estão implementadas neste método?

Liste cada regra encontrada.

Checklist

- Regras identificadas
- Regras separadas
- Linguagem compreensível

Exemplo Real

Método selecionado:

```
void TPedido::CalcularTotal()
{
    Total = ValorProdutos + Frete;

    if (ClienteVIP)
        Total *= 0.9;
}
```

Pergunta:

Explique as regras de negócio.

Resposta esperada:

Regra 1:

Somar produtos e frete.

Regra 2:

Aplicar desconto para clientes VIP.

Exercício 4

Mapeamento de Dependências

Selecione uma classe.

Pergunte:

Liste todas as dependências desta classe.

Informe:

- *Units utilizadas*
 - *Bibliotecas utilizadas*
 - *Classes relacionadas*
-

Checklist

- Units identificadas
 - Dependências externas encontradas
 - Classes relacionadas listadas
-

Exercício 5

Encontrando Problemas

Pergunte:

Analise este código.

Liste:

- *Problemas de manutenção*
 - *Problemas de arquitetura*
 - *Problemas de legibilidade*
 - *Melhorias possíveis*
-

Checklist

- Problemas encontrados
 - Sugestões úteis
 - Melhorias viáveis
-

Analisando Projetos Grandes

Quando a Unit for muito grande:

NÃO pergunte:

Explique tudo.

Pergunte:

Explique apenas:

- Classe principal

- Responsabilidades
- Dependências

Depois aprofunde gradualmente.

Exercício 6

Gerando Documentação Técnica

Selecione uma classe.

Pergunte:

Gere documentação técnica completa.

Inclua:

- *Objetivo*
 - *Responsabilidades*
 - *Métodos públicos*
 - *Métodos privados*
 - *Dependências*
-

Checklist

- Documentação criada
 - Linguagem técnica
 - Fácil entendimento
-

Exercício 7

Criando Diagramas Textuais

Pergunte:

Explique o fluxo desta classe utilizando um diagrama textual.

Exemplo esperado:

Usuário

↓

Tela

↓

Controller

↓

Banco

↓

Resultado

Checklist

- Fluxo compreendido
- Dependências identificadas
- Processo documentado

Modernização de Código

Um dos recursos mais úteis do KAI.

Exercício 8

Selecione uma classe antiga.

Pergunte:

Analise compatibilidade com C++Builder 13.1 Florence.

Liste:

- *Problemas*
- *Riscos*
- *Melhorias*

Checklist

- Unicode
- Clang
- APIs antigas
- Componentes antigos

Exercício 9

Refatoração Controlada

Pergunte:

Refatore este método.

Mantenha exatamente a mesma regra de negócio.

Melhore apenas:

- *Legibilidade*
 - *Organização*
 - *Comentários*
-

Checklist

- Regra mantida
 - Código melhorado
 - Sem alteração funcional
-

Exercício 10

Identificando Código Duplicado

Pergunte:

Existe duplicação de código nesta classe?

Sugira reutilização.

Checklist

- Duplicações encontradas
 - Sugestões úteis
 - Refatoração possível
-

Analisando Performance

Pergunte:

Analise possíveis gargalos de performance.

Liste:

- *Loops desnecessários*
 - *Objetos excessivos*
 - *Alocações repetidas*
-

Checklist

- Gargalos identificados
 - Sugestões coerentes
-

Segurança

Pergunte:

Analise possíveis vulnerabilidades.

Verifique:

- *Buffer overflow*
 - *SQL Injection*
 - *Validações ausentes*
 - *Tratamento de exceções*
-

Checklist

- Problemas encontrados
 - Sugestões apresentadas
-

Exercício Final

Escolha uma Unit real do seu projeto.

Execute os seguintes prompts:

Passo 1

Explique a responsabilidade desta Unit.

Passo 2

Liste dependências.

Passo 3

Liste regras de negócio.

Passo 4

Gere documentação.

Passo 5

Analise melhorias.

Passo 6

Analise compatibilidade com Florence.

Checklist de Aprovação

- Unit compreendida
 - Dependências identificadas
 - Regras documentadas
 - Melhorias propostas
 - Compatibilidade analisada
 - Documentação gerada
-

Cenário Profissional

Imagine um cliente enviando um projeto de C++Builder 6.

Antes de qualquer migração você poderá utilizar o KAI para:

- Entender a arquitetura
- Mapear riscos
- Identificar dependências
- Gerar documentação
- Planejar a migração

Economizando muitas horas de análise manual.

Resumo do Módulo

Neste módulo aprendemos:

- Como entender projetos existentes
- Como mapear dependências
- Como identificar regras de negócio
- Como gerar documentação
- Como localizar problemas
- Como modernizar código
- Como preparar sistemas para migração

No próximo módulo iremos utilizar o KAI para realizar um dos cenários mais importantes para clientes Embarcadero:

Migração de projetos antigos para C++Builder 13.1 Florence.

MÓDULO 6 - MIGRAÇÃO ASSISTIDA DE PROJETOS

C++BUILDER 6 PARA C++BUILDER 13.1

FLORENCE UTILIZANDO KAI

Objetivos

Ao final deste módulo o aluno será capaz de:

- Utilizar o KAI para analisar projetos legados
- Identificar incompatibilidades de compilação
- Preparar projetos para Florence
- Criar um plano de migração
- Modernizar código ANSI
- Identificar problemas de Unicode
- Adaptar projetos para Clang
- Documentar todo o processo de migração

Introdução

A migração de sistemas legados é uma das atividades mais valiosas dentro do ecossistema Embarcadero.

Muitas empresas possuem aplicações desenvolvidas há:

- 10 anos
- 15 anos
- 20 anos
- 25 anos

Muitas delas ainda executando versões antigas como:

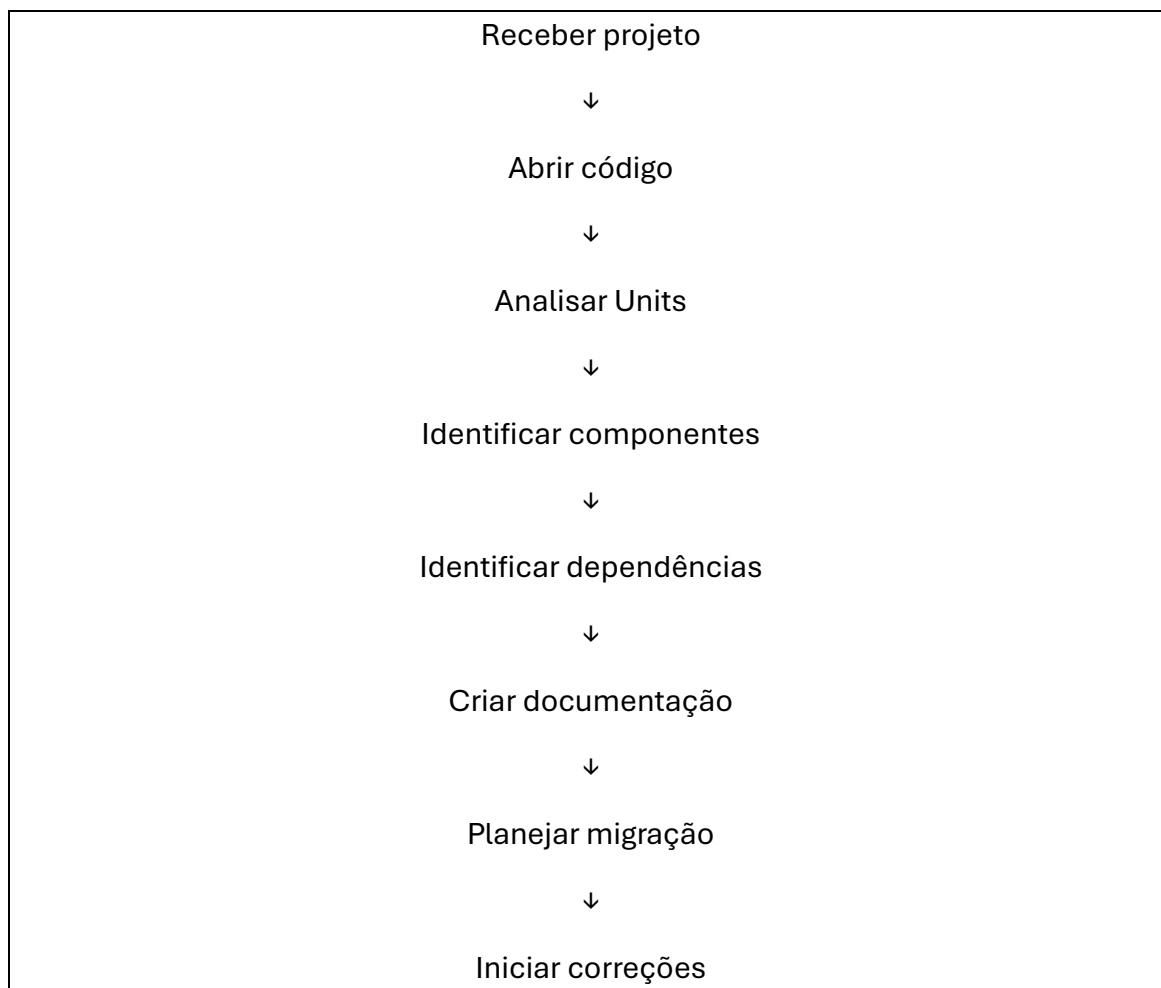
- C++Builder 5
- C++Builder 6
- C++Builder 2007

A principal vantagem do KAI neste cenário não é realizar a migração sozinho.

Sua principal função é acelerar a análise técnica do projeto.

O Processo Tradicional

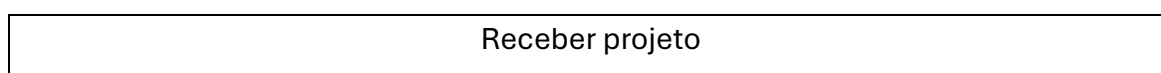
Antes do KAI:

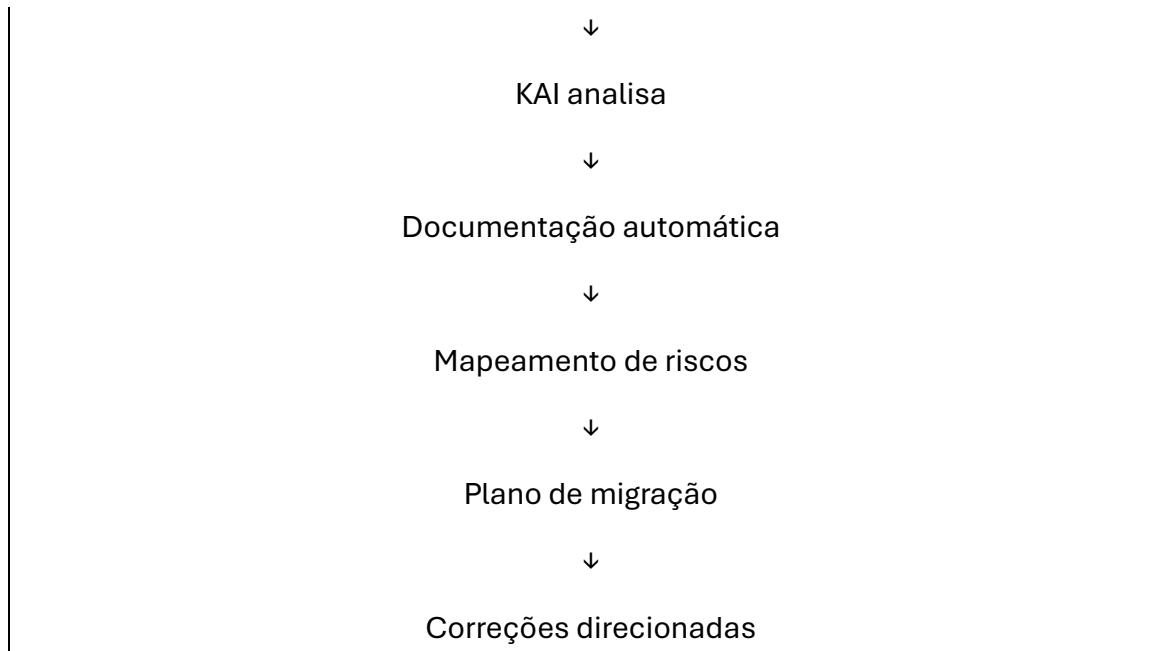


Tempo médio:

Dias ou semanas.

Processo Utilizando KAI





Tempo médio:

Horas ou poucos dias.

Fase 1 - Inventário do Projeto

O primeiro passo nunca é compilar.

O primeiro passo é entender.

Exercício 1

Selecione uma Unit.

Pergunte:

Analise esta Unit.

Liste:

- *Objetivo*
 - *Dependências*
 - *Componentes utilizados*
 - *Possíveis riscos de migração*
-

Checklist

- Objetivo identificado
 - Dependências identificadas
 - Componentes encontrados
 - Riscos encontrados
-

Exercício 2

Selecione o arquivo principal.

Pergunte:

Analise este projeto.

Liste:

- *Componentes VCL*
 - *Bibliotecas externas*
 - *APIs Windows*
 - *Dependências críticas*
-

Checklist

- Componentes encontrados
 - APIs encontradas
 - Dependências mapeadas
-

Fase 2 - Compatibilidade com Florence

Pergunta:

Analise compatibilidade com C++Builder 13.1 Florence.

Liste:

- *Problemas*
 - *Melhorias*
 - *Ajustes necessários*
-

Checklist

- Unicode
- Clang

- STL
 - Componentes
 - APIs
-

Problema 1 - Unicode

Um dos problemas mais comuns.

Exemplo legado:

```
char Nome[50];
```

Prompt:

Converta para abordagem compatível com Florence.

Resultado esperado

Uso de:

```
String Nome;
```

Checklist

- Unicode compatível
 - Código atualizado
-

Problema 2 - APIs ANSI

Exemplo:

```
CreateFileA
```

Prompt:

Substitua APIs ANSI por Unicode.

Checklist

- APIs identificadas
 - Unicode aplicado
-

Problema 3 - Conversões de Texto

Pergunte:

Localize possíveis problemas de conversão ANSI/Unicode.

Checklist

- Conversões encontradas
 - Soluções sugeridas
-

Fase 3 - Compatibilidade com Clang

Florence utiliza compilador moderno.

Muitos projetos antigos utilizam recursos incompatíveis.

Exercício 3

Selecione uma classe.

Pergunte:

Analise compatibilidade com Clang.

Checklist

- Conversões implícitas
- Casts perigosos
- Includes antigos

☐ Templates problemáticos

Exercício 4

Pergunte:

Modernize esta classe para padrões modernos de C++.

Checklist

- Código atualizado
- Legibilidade melhorada

- Compatível com Florence

Fase 4 - Componentes de Terceiros

Muitos projetos utilizam:

- RxLib
- TurboPower
- JVCL
- QuickReport
- FastReport
- Componentes proprietários

Exercício 5

Pergunte:

Identifique componentes de terceiros utilizados nesta Unit.

Checklist

- Componentes encontrados
- Dependências listadas
- Possíveis substituições sugeridas

Fase 5 - Planejamento da Migração

Prompt:

Crie um plano de migração para este projeto.

Divida em etapas.

Resultado Esperado

Etapas 1

Inventário

Etapas 2

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

Compatibilidade

Etapas 3

Correções Unicode

Etapas 4

Correções Clang

Etapas 5

Testes

Etapas 6

Entrega

Checklist

- Plano criado
- Etapas coerentes
- Priorização correta

Fase 6 - Geração de Relatório

Prompt:

Crie um relatório executivo de migração.

Inclua:

- *Riscos*
- *Complexidade*
- *Tempo estimado*
- *Recomendações*

Checklist

- Relatório criado
 - Linguagem executiva
 - Fácil entendimento
-

Exercício Final - Utilize um projeto real.

Execute:

Prompt 1

Analise arquitetura.

Prompt 2

Liste dependências.

Prompt 3

Liste riscos.

Prompt 4

Analise compatibilidade Florence.

Prompt 5

Sugira modernizações.

Prompt 6

Crie plano de migração.

Prompt 7

Gere relatório executivo.

Checklist Final

- Arquitetura documentada
 - Dependências identificadas
 - Problemas encontrados
 - Compatibilidade analisada
 - Plano criado
 - Relatório gerado
-

Estudo de Caso - Migração Assistida

Projeto:

C++Builder 6

↓

Análise com KAI

↓

Mapeamento

↓

Planejamento

↓

Correções

↓

Florence

↓

Validação

Boas Práticas –

Nunca solicite:

Migre todo o projeto.

Prefira:

Analise esta Unit.

Depois:

Analise este Form.

Depois:

Crie plano de migração.

O resultado será muito melhor.

Encerramento do Curso

Você concluiu o treinamento:

KAI para C++Builder 13.1 Florence

Ao longo dos módulos você aprendeu:

- ✓ Conceitos fundamentais do KAI
- ✓ Instalação e configuração
- ✓ Engenharia de Prompt
- ✓ Geração de código VCL
- ✓ Análise de projetos reais
- ✓ Documentação automática
- ✓ Modernização de código
- ✓ Migração assistida para Florence
- ✓ Boas práticas profissionais

RAD Studio 13.1 Florence Autor: Thiago Santos Embarcadero MVP Versão 1.0

KAI + ESP32

Módulo Avançado 5

Construindo Ferramentas MCP em C++Builder

Conteúdo:

Fundamentos, Configuração, Prompts, VCL, Análise de Código e Migração Assistida.